

PROGRESSO TOTAL

5 de 5

100% das etapas executadas

DURAÇÃO GLOBAL

2h 50min

Início 15:22 • Conclusão 18:11

REDUÇÃO DO MONÓLITO

-98.1%

app.py: 3.340 para 64 linhas

BANCO PADRONIZADO

Postgres

Supabase Pooler IPv4 único

1. Cronograma de Execução e Durações

Etapa	Objetivo Estratégico	Horário	Duração	Commit Git
Passo 1	Segurança imediata, saneamento de chaves e protocolos de governança	15:22 – 15:28	~6 min	77743c5
Passo 2	Modularização em 13 Blueprints Flask e agente anti-monólito (<500 linhas)	15:29 – 16:14	~45 min	6180080
Passo 3	Aposentadoria de adaptações SQL e adoção nativa do PostgreSQL Supabase	16:16 – 17:01	~45 min	74bb309
Passo 4	Modernização leve com HTMX, Alpine.js e gráficos analíticos (Chart.js)	17:01 – 17:18	~18 min	c41acec
Passo 5	Telemetria Vercel (/infra/performance) e Alerta WhatsApp Meta aos 75%	17:26 – 18:11	~45 min	PRONTO DEPLOY

2. Síntese Técnica das Implementações

 Passo 1 — Segurança Imediata (Prioridade Zero)

Duração: ~6 min

Eliminação de vulnerabilidades e estabelecimento de esteira segura de entrega contínua.

- Saneamento de credenciais e tokens em arquivos de código;
- Remoção definitiva de automações legadas instáveis que causavam travamentos;
- Implementação dos comandos oficiais de governança [start] e [deploy].

 Passo 2 — Modularização em Blueprints (Arquitetura Limpa)

Duração: ~45 min

Fatiamento estrutural do monólito com limite rígido de 500 linhas por arquivo.

- Criação de 13 módulos coesos em routes/ (vendas, fretes, estoque, CRM, importação, etc.);
- Redução drástica do app.py de ~3.340 linhas para 64 linhas;
- Instalação do agente auditor automático scripts/monolith_watcher.py.

 Passo 3 — Padronização de Banco (PostgreSQL Oficial Supabase)

Duração: ~45 min

Aposentadoria de compatibilidades SQLite e uso de recursos avançados do PostgreSQL.

- Eliminação do conversor frágil .replace("?", "%s") em database.py;
- Queries nativas com parâmetros tipados, RETURNING e funções de agregação;
- Conexão de altíssimo desempenho via Pooler IPv4 oficial Supabase em porta 6543.

 Passo 4 — Modernização Leve de Frontend (HTMX + Alpine + Dashboards)

Duração: ~18 min

Reatividade moderna e fluida sem a complexidade de reconstruir tudo em React/Vue.

- Kanban do CRM e filtros de estoque atualizam instantaneamente sem recarregar tela;
- Dashboards analíticos enriquecidos com gráficos dinâmicos e responsivos (Chart.js);
- Preservação integral dos templates Jinja2 e performance ultra leve no carregamento.

 Passo 5 — Hospedagem Futura e Monitor de Desempenho Vercel

Duração: ~45 min

Visibilidade em tempo real das cotas da Vercel e gatilho inteligente de migração.

- Painel /infra/performance restrito exclusivamente a Jam e Fauzer;
- Acompanhamento dos 4 limites Hobby: GB-Horas, Bandwidth, Invocação/dia e Latência;
- Gatilho automático aos 75% com envio de alerta no WhatsApp via Meta API e botão de teste.